

Machine Learning Neural Networks

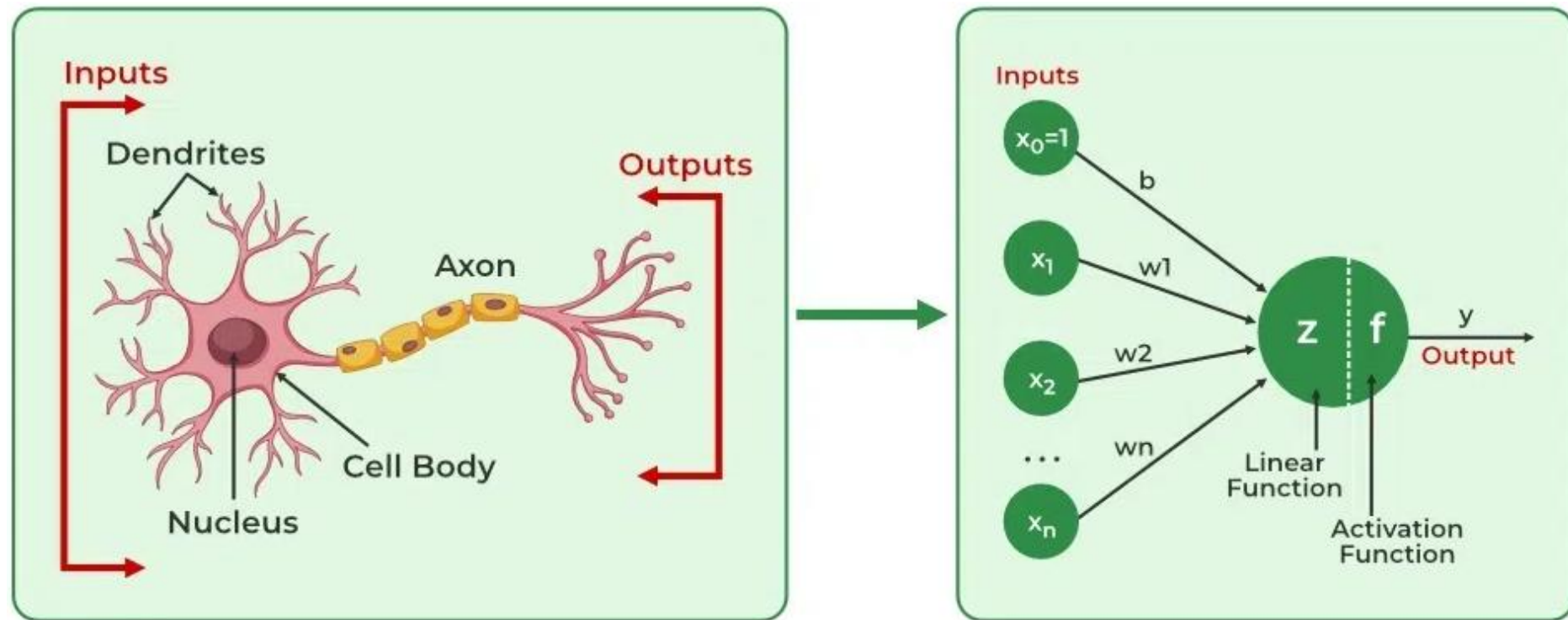
Dr. Ahmed Aljarah

Neural networks

- Neural networks are machine learning models inspired by the structure and operation of the human brain.
- They are designed to mimic how the brain processes information and perform similar tasks.
- They are composed of interconnected nodes, called neurons, that work together to :
 - Process information
 - Learn patterns from data
 - Pattern recognition
 - Decision-making

From brain to machine neural networks

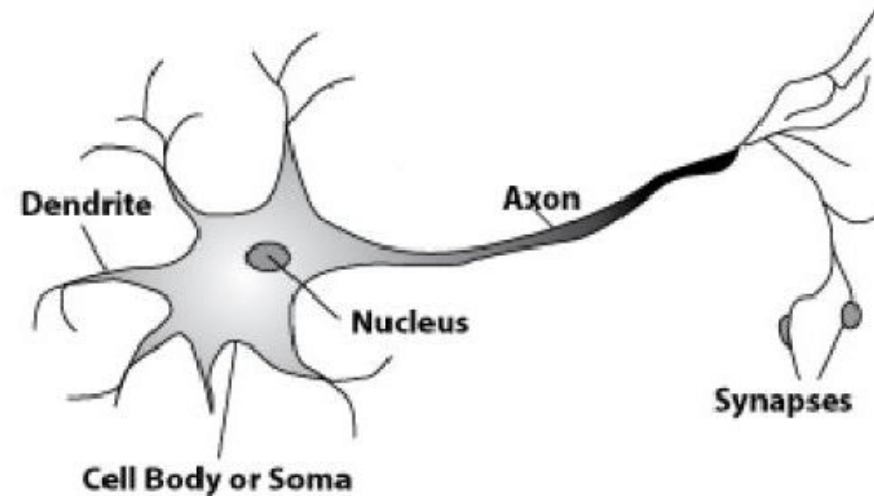
Neural networks are computational models that emulate the structure of biological neural systems



Biological Neuron (Brain)

A neuron in the brain has several main parts:

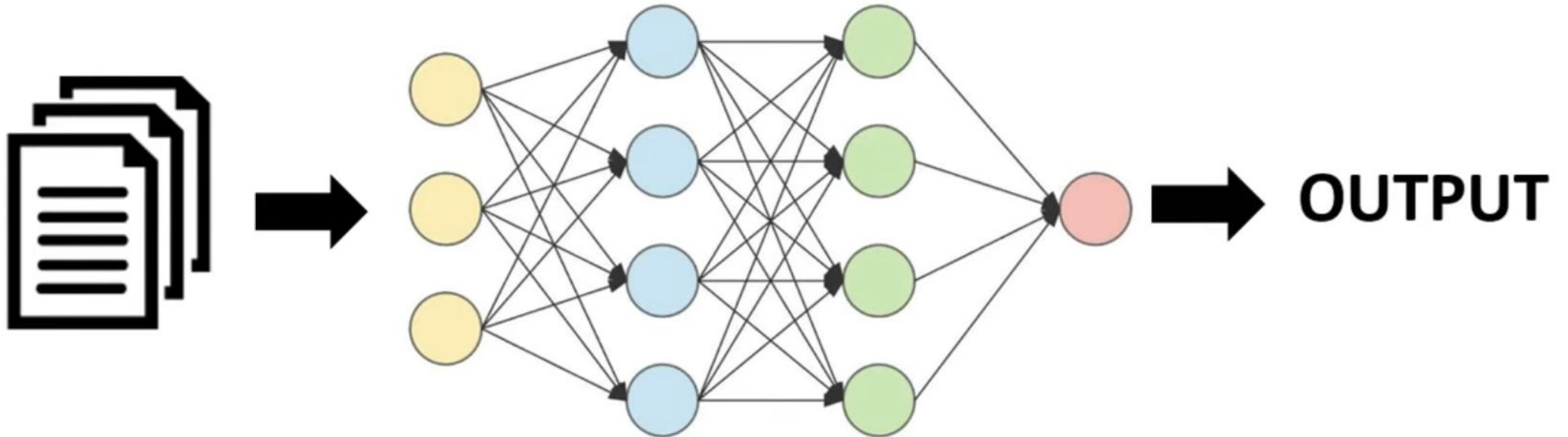
- Dendrites receive signals (inputs) from other neurons
- Cell body processes the received information
- Nucleus controls neuron activity
- Axon carries the signal forward
- A synapse is the connection point between two neurons in the brain. It is where the output signal from one neuron is transmitted to another neuron
- The outputs send information through the synapse to communicate with other neurons
- The biological neuron receives information, processes it, and produces an output signal



Human brain



Neural networks in ML

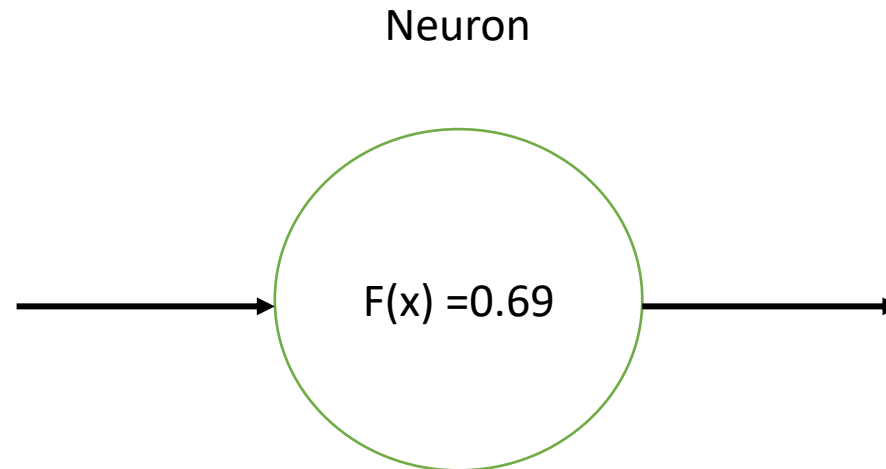


Neural networks components

- Neurons: The basic units that receive inputs, calculate $f(z)$ and apply an activation function.
- Connections: Links between neurons that carry information, regulated by weights and biases.
- Weights and Biases: These parameters determine the strength and influence of connections.
- Propagation functions: Mechanisms that help process and transfer data across layers of neurons.
- Learning rule: The method that adjusts weights and biases over time to improve accuracy.

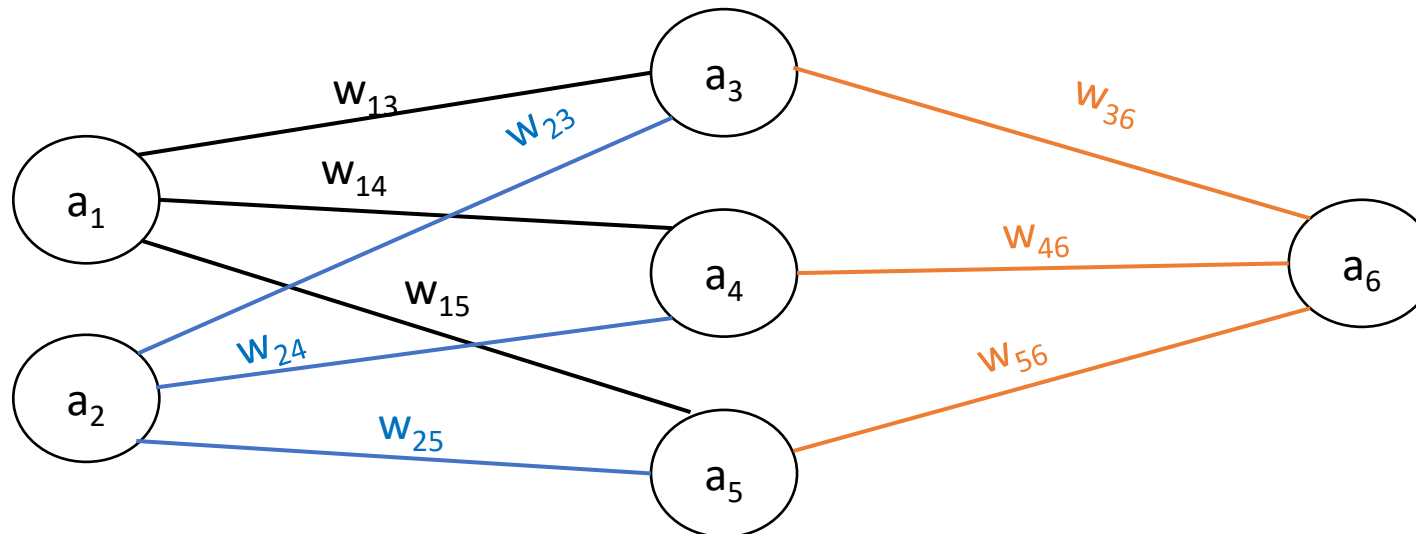
Neural networks

- Neural networks are systems composed of processing elements (PEs) along with the connections and communication between them.
- A processing element is a simplified representation of a biological neuron, commonly referred to as an artificial neuron. Each PE receives input data, processes it, and generates an output.
- Neuron
 - Receives inputs
 - Multiplies by weights
 - Adds bias
 - Applies activation
 - function Produces output



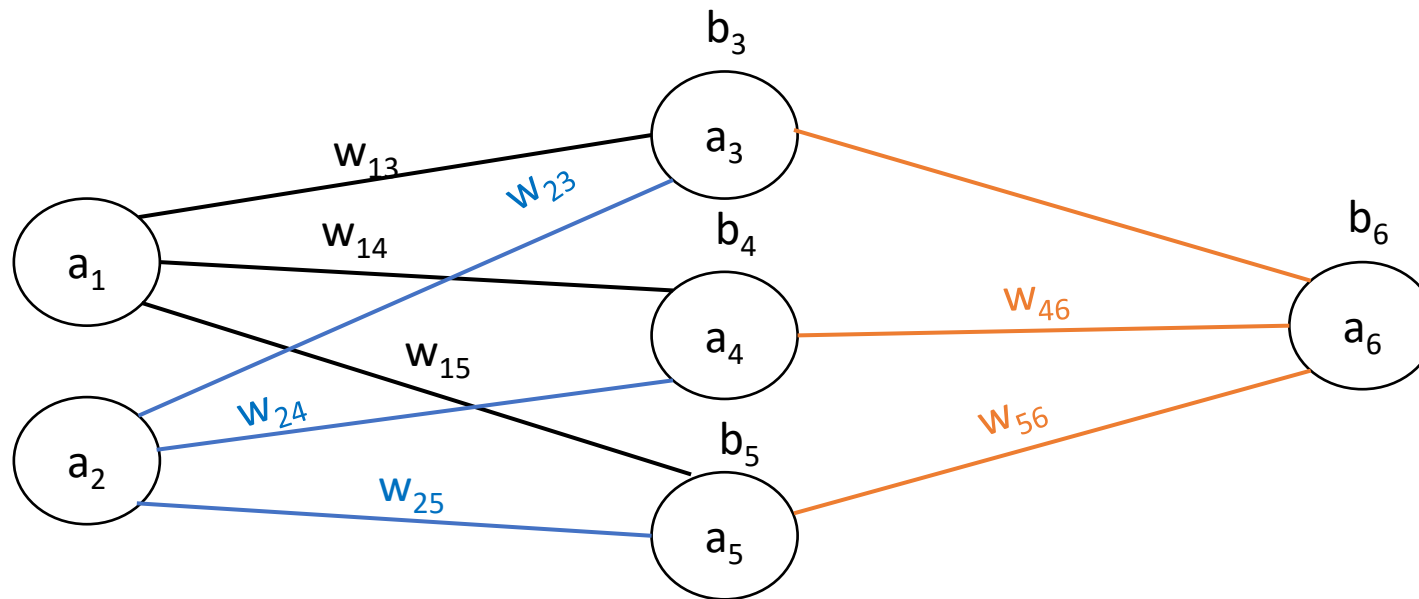
Connections

- Connections, also known as synaptic links, are the pathways that connect neurons in a neural network.
- They allow information to flow from one neuron to another.
- Each connection has a weight, that determines the importance of the signal.
- During learning, these weights are adjusted to improve the network's performance.



Weights and Biases

- Weights (w): control the importance of each input
- Bias (b): shifts the output value
- Help the model adjust and learn patterns
- Updated during training to reduce error



Learning rule

- The learning rule defines how a neural network improves by updating its weights and biases based on the error.
- Compares predicted output with the correct output
- Calculates the error
- Adjusts weights and biases to reduce the error
- Repeats until the model improves

Neural networks layers

1. Input Layer:

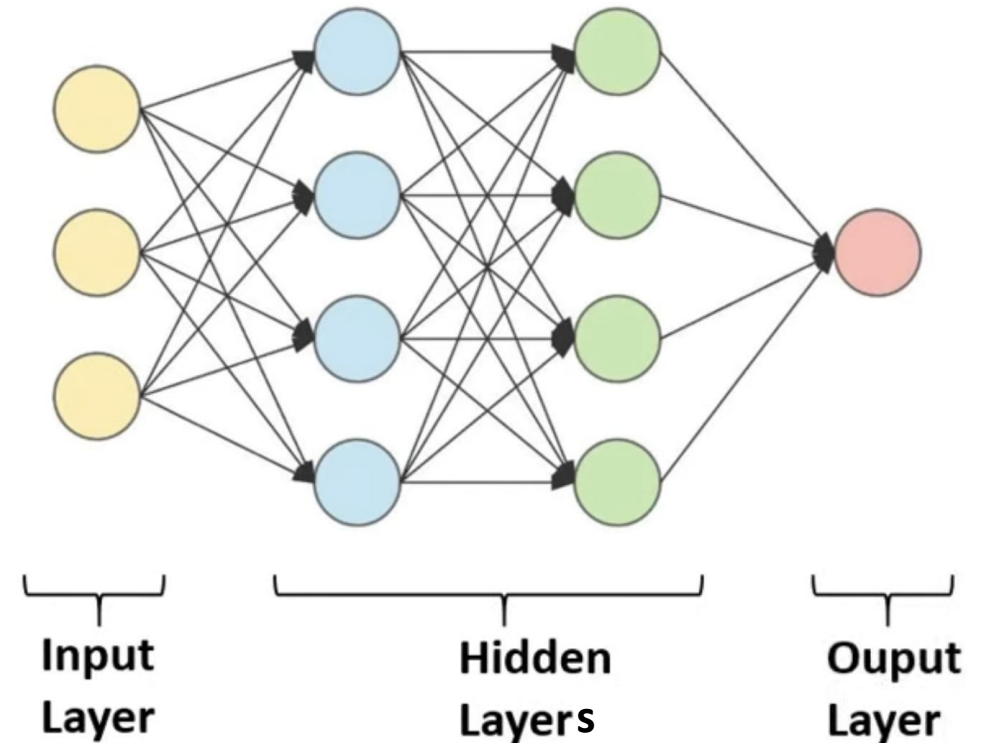
- Entry point of the network
- Receives raw data (features)
- Each neuron = one input feature

2. Hidden Layers:

- Perform the main computations
- Learn patterns and relationships in data
- Each layer transforms the data step by step
- Can have one or multiple layers

3. Output Layer:

- Produces the final prediction
- Format depends on the task, Classification or Regression



Propagation functions

Forward propagation:

- Forward propagation define how data moves and is processed inside a neural network.
- Combine inputs using weights and bias
- Compute a value called net input (z)
- Pass the result to the activation function(Sigmoid, Tanh or ReLU)
- Send the output to the next layer

Forward propagation

Forward propagation is the process in a neural network where input data moves through the network layer by layer to produce a prediction.

Input → Hidden → Output

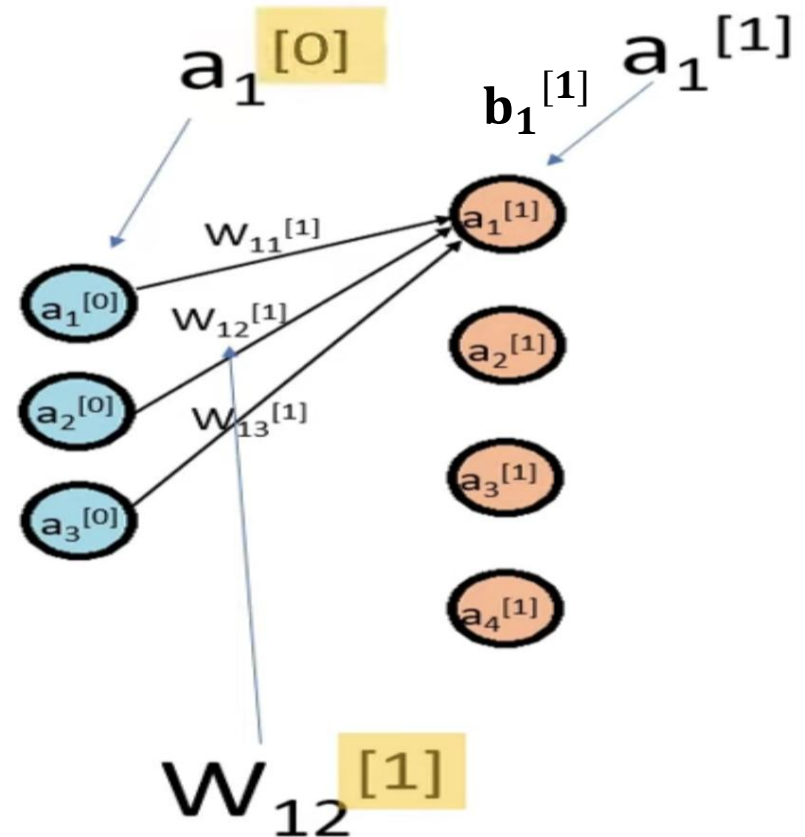
The model:

- Takes input data
- Processes it through layers

$$z^{[l]} = W^{[l]} * a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = f(z^{[l]})$$

- Produces an output (prediction)
- Then we compare:
 - Predicted value vs correct answer
 - This difference is called the error (loss)



Forward propagation

From input Layer
to hidden Layer1

$$z_1 = w_1 * a_0 + B_1$$
$$a_1 = f(z_1)$$

From hidden
Layer1 to hidden
Layer2

$$z_2 = w_2 * a_1 + B_2$$
$$a_2 = f(z_2)$$

From hidden
Layer2 to output
Layer

$$z_3 = w_3 * a_2 + B_3$$
$$a_3 = f(z_3) = y_pred$$

Random weights

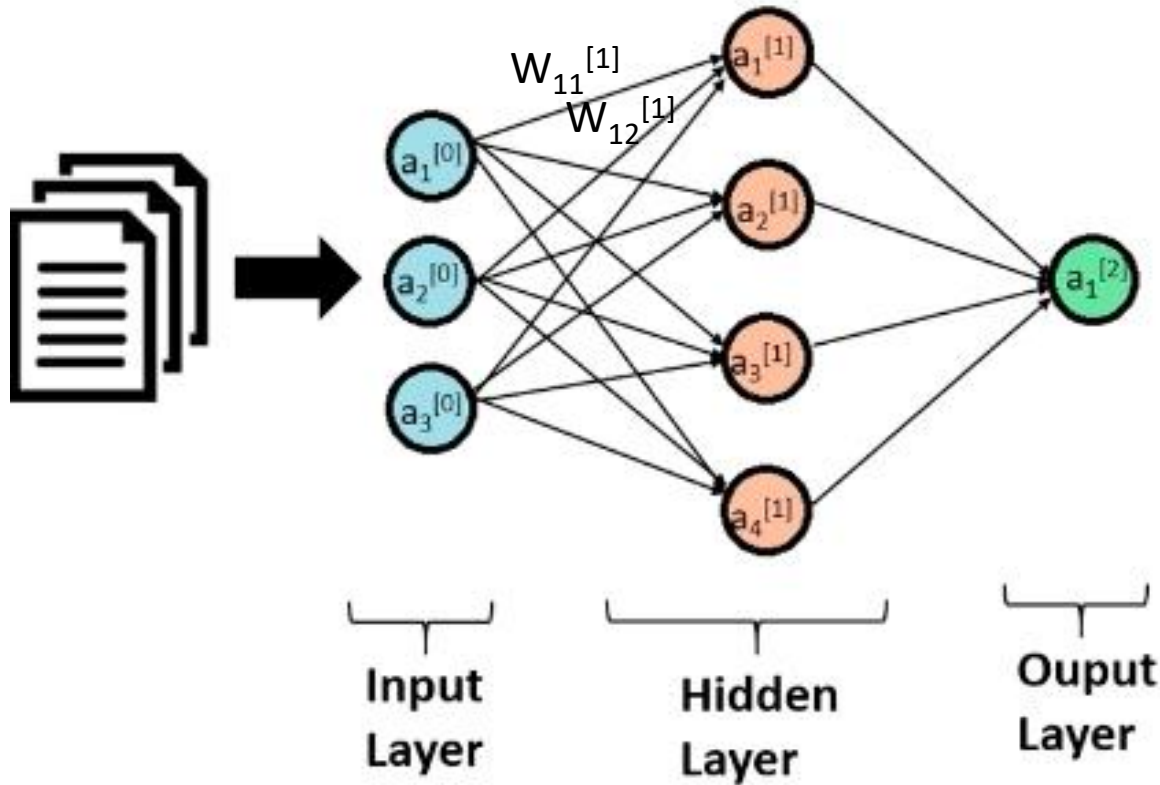


After training



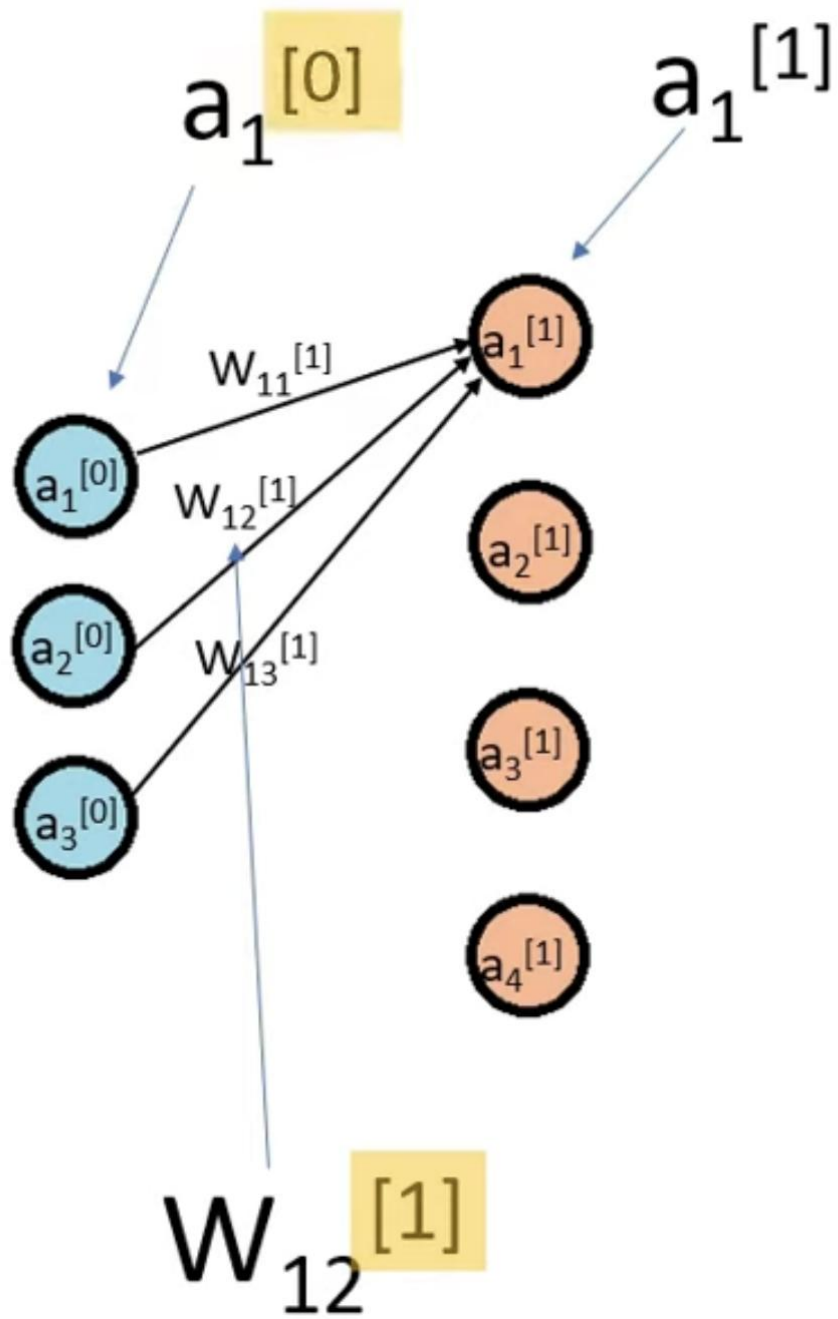
Trained weights

Forward propagation



$$a_1^{[1]} = \text{activation function}(w_{11}^{[1]} * a_1^{[0]} + w_{12}^{[1]} * a_2^{[0]} + w_{13}^{[1]} * a_3^{[0]} + B1)$$

$$a_2^{[1]} = \text{activation function}(w_{21}^{[1]} * a_1^{[0]} + w_{22}^{[1]} * a_2^{[0]} + w_{23}^{[1]} * a_3^{[0]} + B1)$$



$$a_1^{[1]} = \text{activation_function}(\\ W_{11}^{[1]} * a_1^{[0]} + \\ W_{12}^{[1]} * a_2^{[0]} + \\ W_{13}^{[1]} * a_3^{[0]} + \\ B1)$$

Activation functions Types

Output layer:

- Binary classification: Sigmoid function
- Multi-class classification: Softmax function
- Regression: no need for activation function on the output neuron

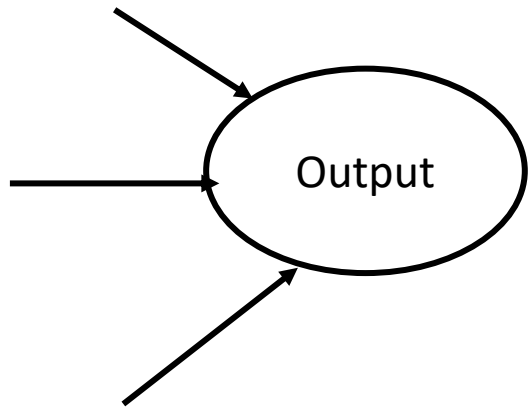
Hidden layers:

- Sigmoid function
- Tanh Function
- Relu (Rectified Linear Unit) Function

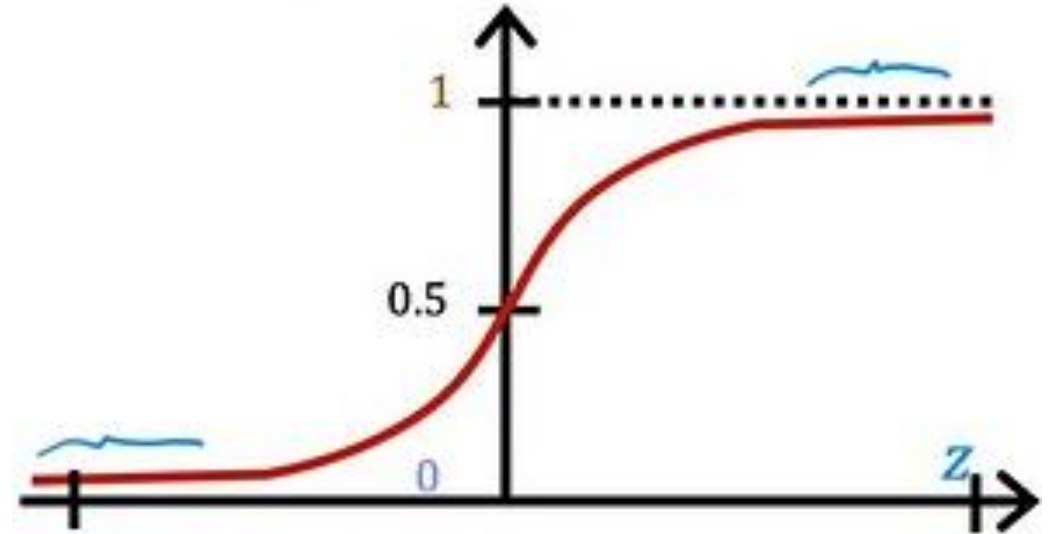


Sigmoid function

$$f(z) = \frac{1}{1+e^{-z}}$$



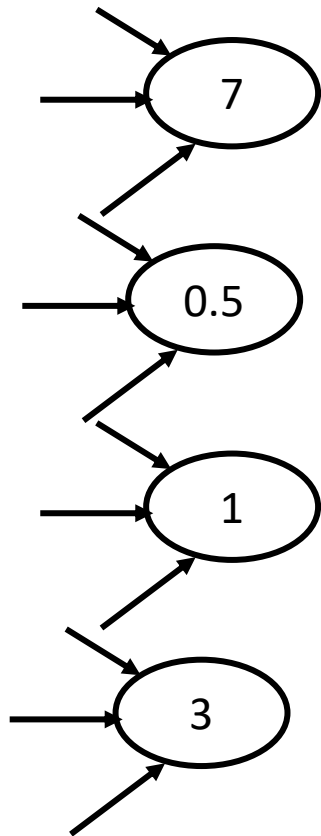
$$\begin{aligned} &\rightarrow f(z) \geq 0.5 \rightarrow 1 \\ &f(z) < 0.5 \rightarrow 0 \end{aligned}$$



Softmax function

$$\text{Softmax} = f_i(z) = \frac{e^{z_i}}{\sum_{i=1}^k e^{z_i}}$$

Example:



$$F(7) = \frac{e^7}{e^7 + e^{0.5} + e^1 + e^3} = \frac{1096.6331}{1121.08} = 0.9782$$

$$F(0.5) = \frac{e^{0.5}}{e^7 + e^{0.5} + e^1 + e^3} = \frac{1.6487}{1121.08} = 0.00147$$

$$F(1) = \frac{e^1}{e^7 + e^{0.5} + e^1 + e^3} = \frac{2.7182}{1121.08} = 0.00242$$

$$F(3) = \frac{e^3}{e^7 + e^{0.5} + e^1 + e^3} = \frac{20.0855}{1121.08} = 0.0179$$

$z=[z_1, z_2, \dots, z_k]$ are just scores

Softmax turns them into:

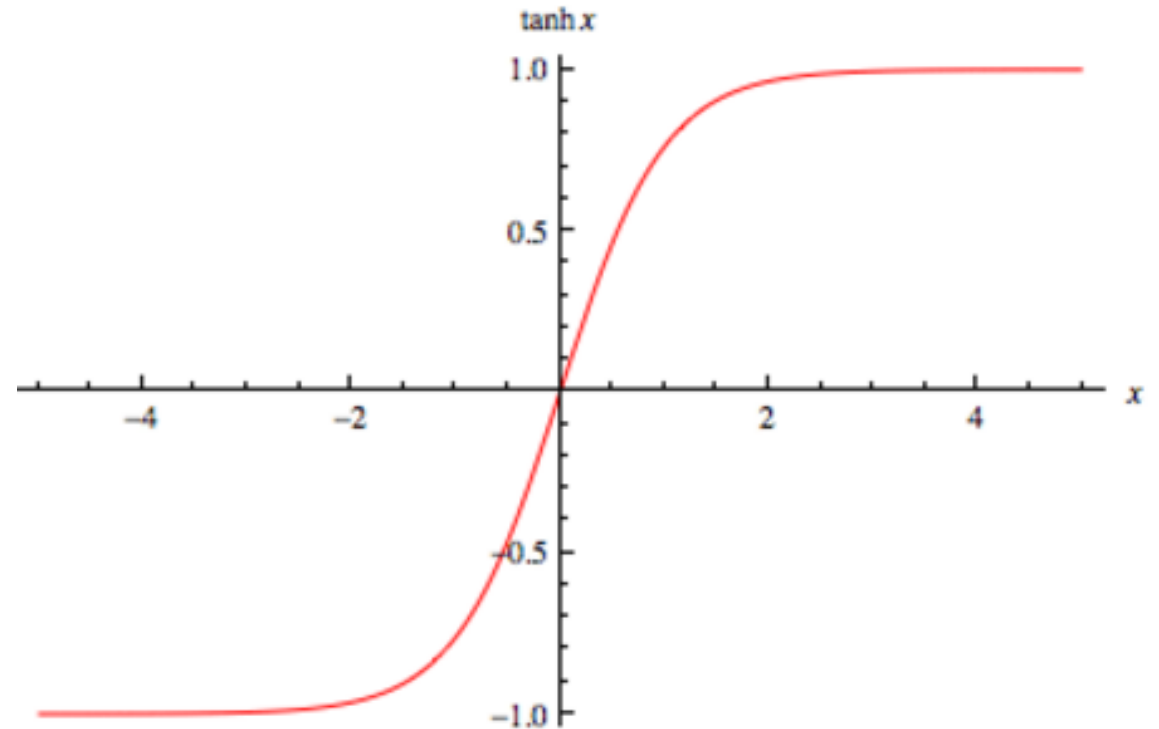
$Y_{\text{pred}}=[p_1, p_2, \dots, p_k]$

k is total number of classes

Tanh function

$$f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- With tanh values centered around 0
- Better balance
- Faster convergence:
 - Has larger gradients
 - Learning is more efficient



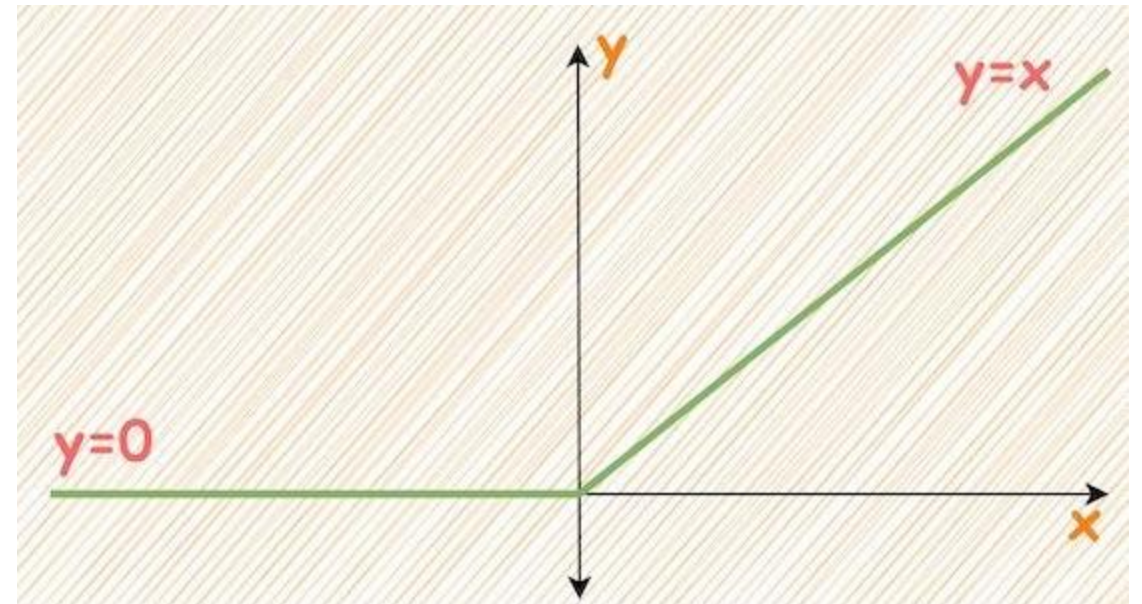
ReLU (Rectified Linear Unit) function

It passes positive values and block negative ones

$$f(z) = \max(0, z) \rightarrow \begin{cases} \text{If } x > 0 \rightarrow \text{output} = z \\ \text{If } x \leq 0 \rightarrow \text{output} = 0 \end{cases}$$

Examples:

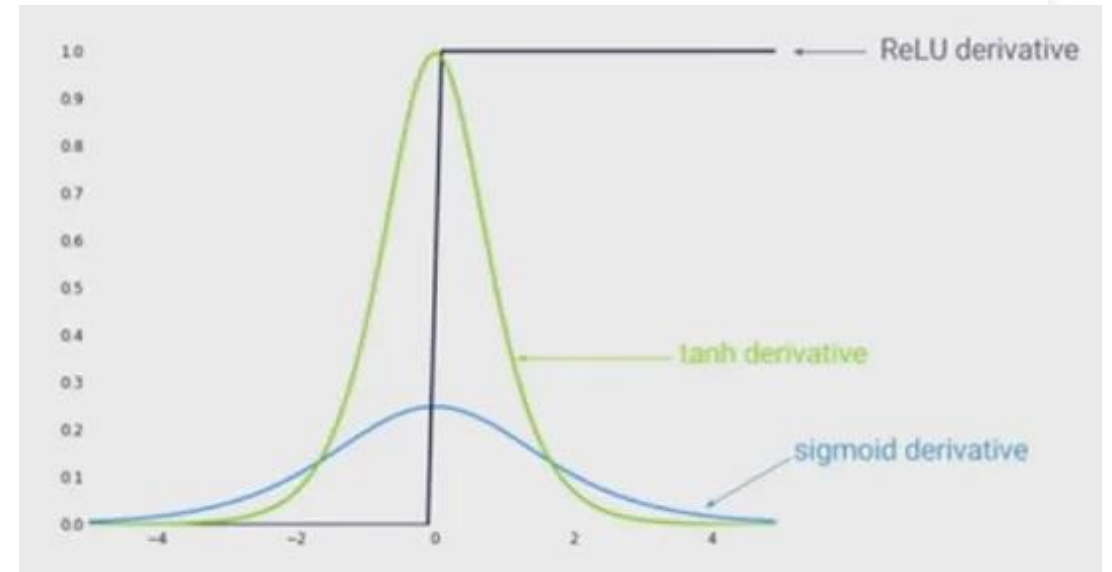
- $\text{ReLU}(5) = 5$
- $\text{ReLU}(-3) = 0$
- $\text{ReLU}(0) = 0$



Piece wise linear

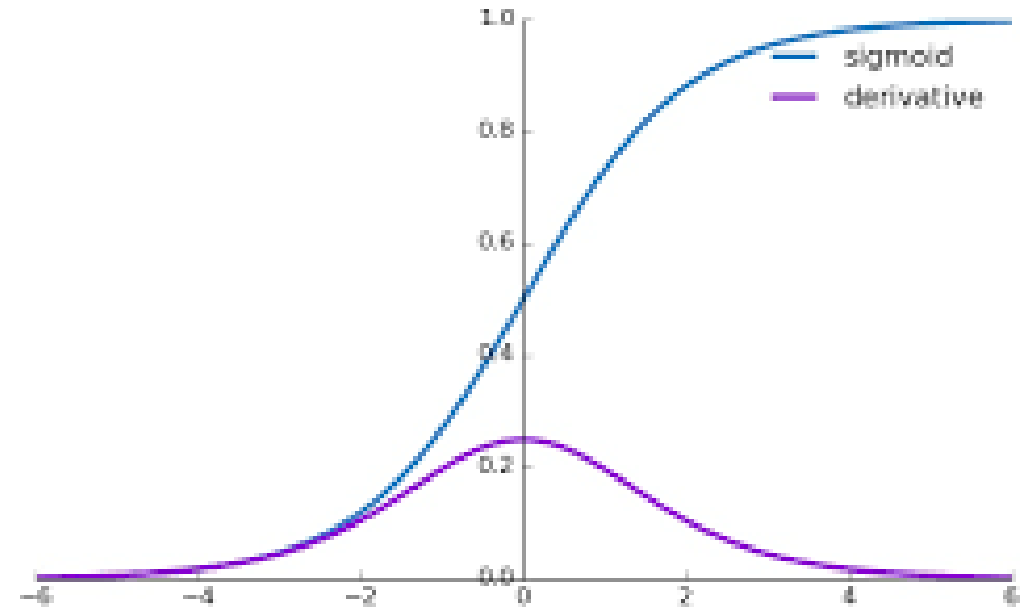
Vanishing gradient problem

- It occurs when gradients become very small during backpropagation, causing slow or no learning, especially in deep neural networks.
- Gradients move backward through many layers.
- Gradients become smaller, making it difficult for deep networks to learn complex patterns.



Sigmoid derivative

- Sigmoid $\rightarrow f(z) = \frac{1}{1+e^{-z}}$
- Derivative $\rightarrow f'(z) = f(z) \cdot (1 - f(z))$
- The maximum derivative of sigmoid is 0.25 this happens at $z=0$
- For very large positive or negative values of z , the derivative becomes close to 0, that contributes to the vanishing gradient problem

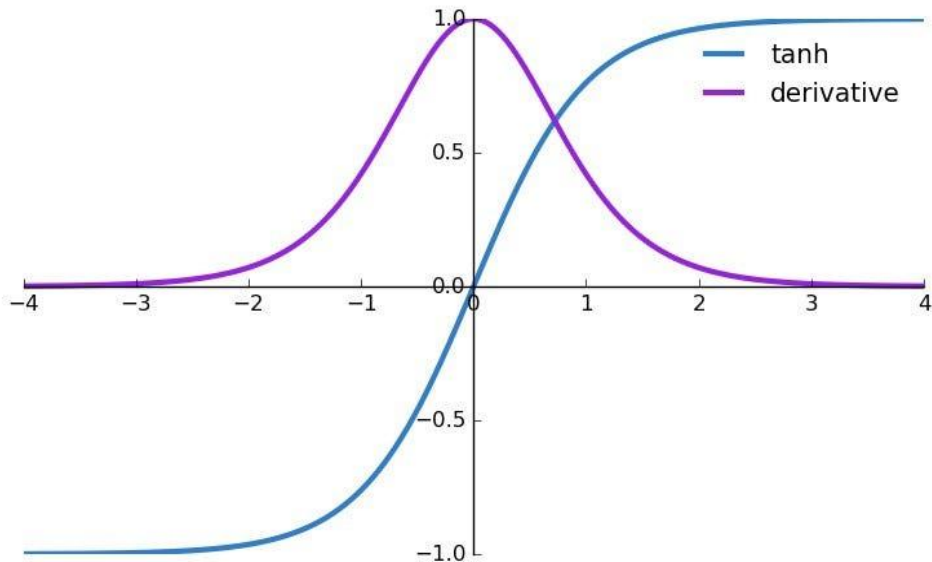


Tanh derivative

$$\text{Tanh} \rightarrow f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\text{Derivative} \rightarrow f'(z) = 1 - (f(z))^2$$

- The maximum of the Derivative is 1.
- Occurs when $f(z)=0$
- Happens at: $z=0$, then: $f'(0)=1-0^2$
- Tanh has larger gradients, which helps learning happen faster than sigmoid



ReLU derivative

ReLU $f(z) = \max(0, z) \rightarrow$

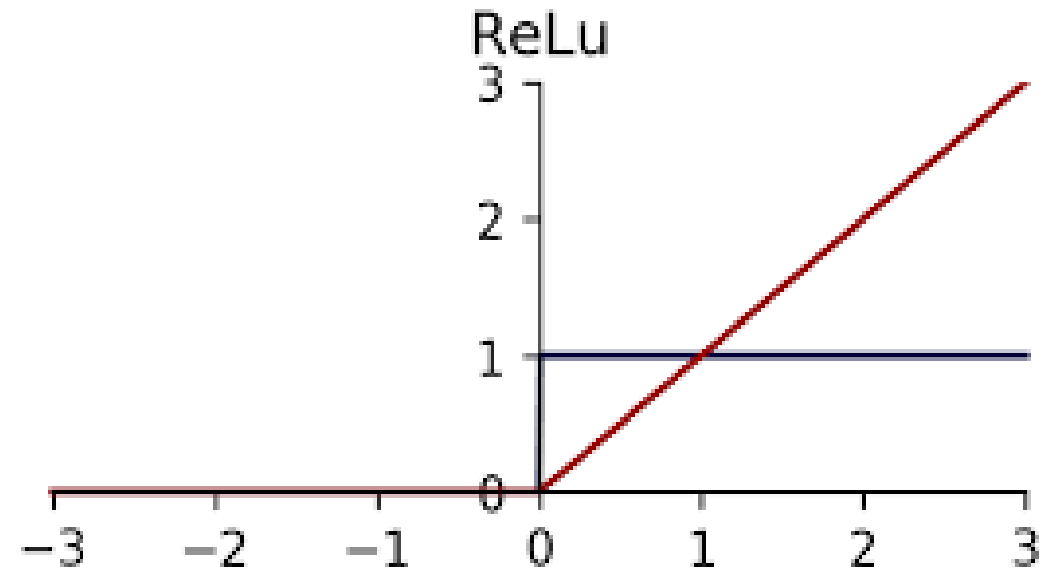
If $x > 0 \rightarrow \text{output} = z$
If $x \leq 0 \rightarrow \text{output} = 0$

Derivative:

$$f'(z_j) = 1 \quad \text{if } z > 0$$

$\rightarrow$

$$f'(z_j) = 0 \quad \text{if } z \leq 0$$



Backpropagation

- Backpropagation calculates how much each weight and bias contributed to the prediction error and then adjust those weights to reduce future errors.
- It works by sending the error backward through the network, from output layer to input layer.
 - Moves backward through the network
 - Measures each layer's responsibility for the error
 - Calculates how to adjust weights and bias to reduce it

Cost function

Regression

Error(Loss) = $|a_i - y_i|$ → for i observation

Cost = $\frac{1}{m} \sum_{i=1}^m |a_i - y_i|$ → *Mean Absolute Error*

Cost = $\frac{1}{2m} \sum_{i=1}^m (a_i - y_i)^2$ → *Mean Squared Error*



$a_i = y_{\text{pred}}$

$y_i = y_{\text{actual}}$

M = number of observation

Binary classification

Error = $-[y_i \log(a_i) - (1 - y_i) \log(1 - a_i)]$

Cost = $-\frac{1}{m} \sum_{i=1}^m [y_i \log(a_i) - (1 - y_i) \log(1 - a_i)]$



$a_i = y_{\text{pred}}$

$y_i = y_{\text{actual}}$

M = number of observation

Multi-class classification

Error = $-\log(a_i)$

Cost = $-\frac{1}{m} \sum_{i=1}^m [y_{i1} \log(a_{i1}) + y_{i2} \log(a_{i2}) + \dots + y_{in} \log(a_{in})]$

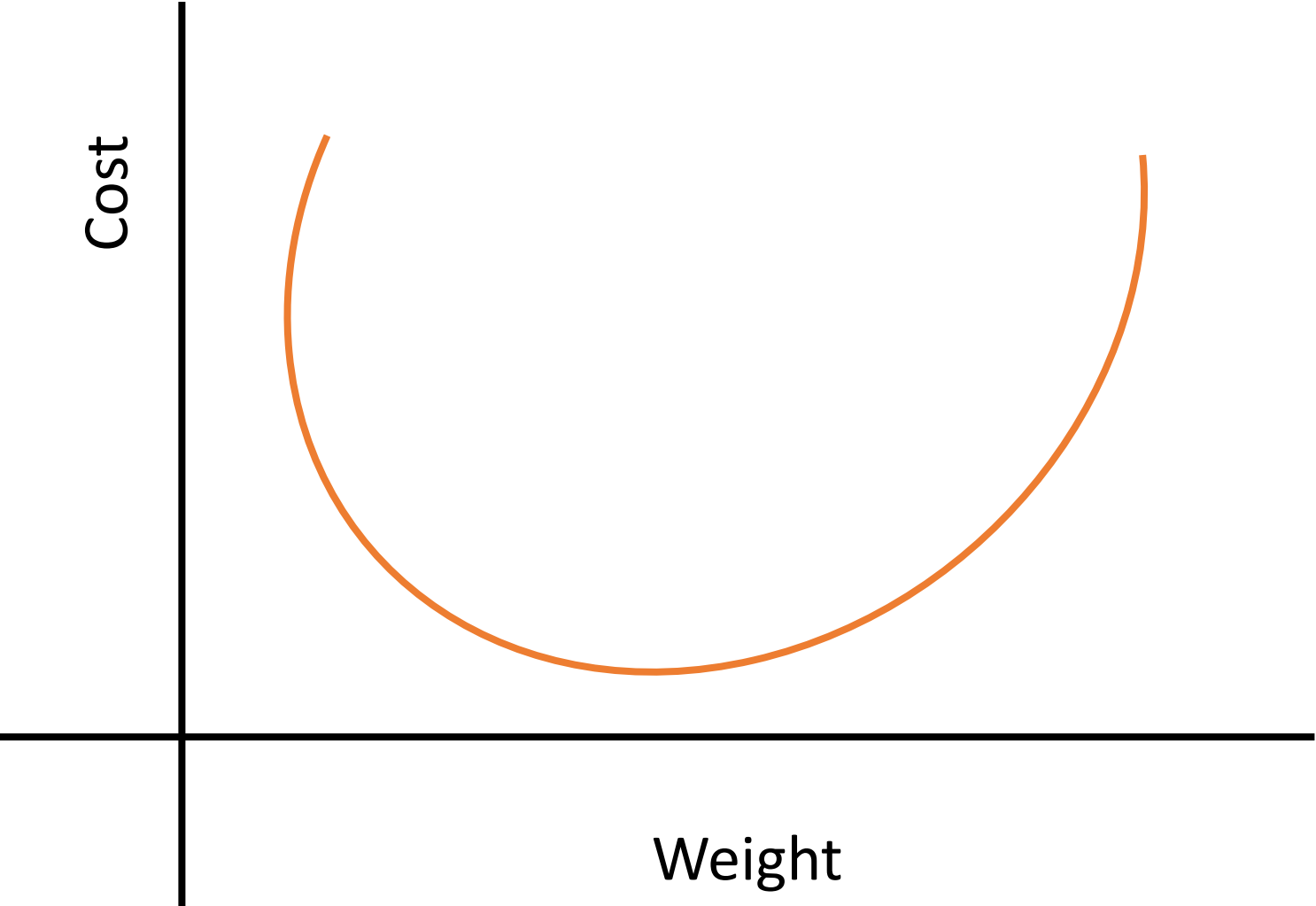


$a_i = y_{\text{pred}}$

N = class number

M = number of observation

Cost



Gradients descent

The change in each weight (update rule) is calculated as:

$$w_{ij}^{\text{new}} = w_{ij}^{\text{old}} + \Delta w_{ij}$$

$$\Delta w_{ij} = \eta \cdot \delta_j \cdot O_i$$

Bias is updated almost the same as weights, but without O_i

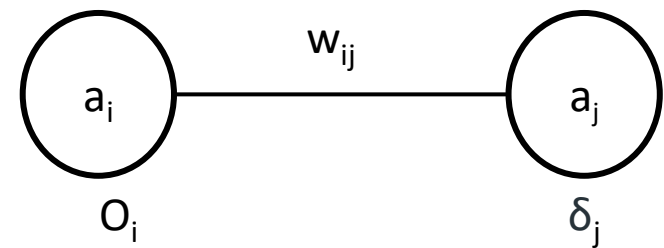
This is because weight depends on input from previous neuron

$$b_j^{\text{new}} = b_j^{\text{old}} + \Delta b_j$$

$$\Delta b_j = \eta \cdot \delta_j$$

Where:

- δ_j is the error term for each unit
- η is the learning rate.
- O_i is the output of neuron
- i is the input to the weight, index of a neuron in the previous layer



δ_j (error term) calculations

Output layer:

$$\delta_j = (y_j - a_j) \cdot f'(z_j)$$

- If $f(z_j)$ is sigmoid:
 $f'(z) = a_j (1 - a_j)$
 $\delta_j = (y_j - a_j) \cdot a_j (1 - a_j)$
- If $f(z_j)$ is tanh:
 $f'(z) = 1 - a_j^2$
 $\delta_j = (y_j - a_j)(1 - a_j^2)$

Where:

- y_j is the target
- a_j is the predicted output
- $f'(z_j)$ is the derivative of activation function

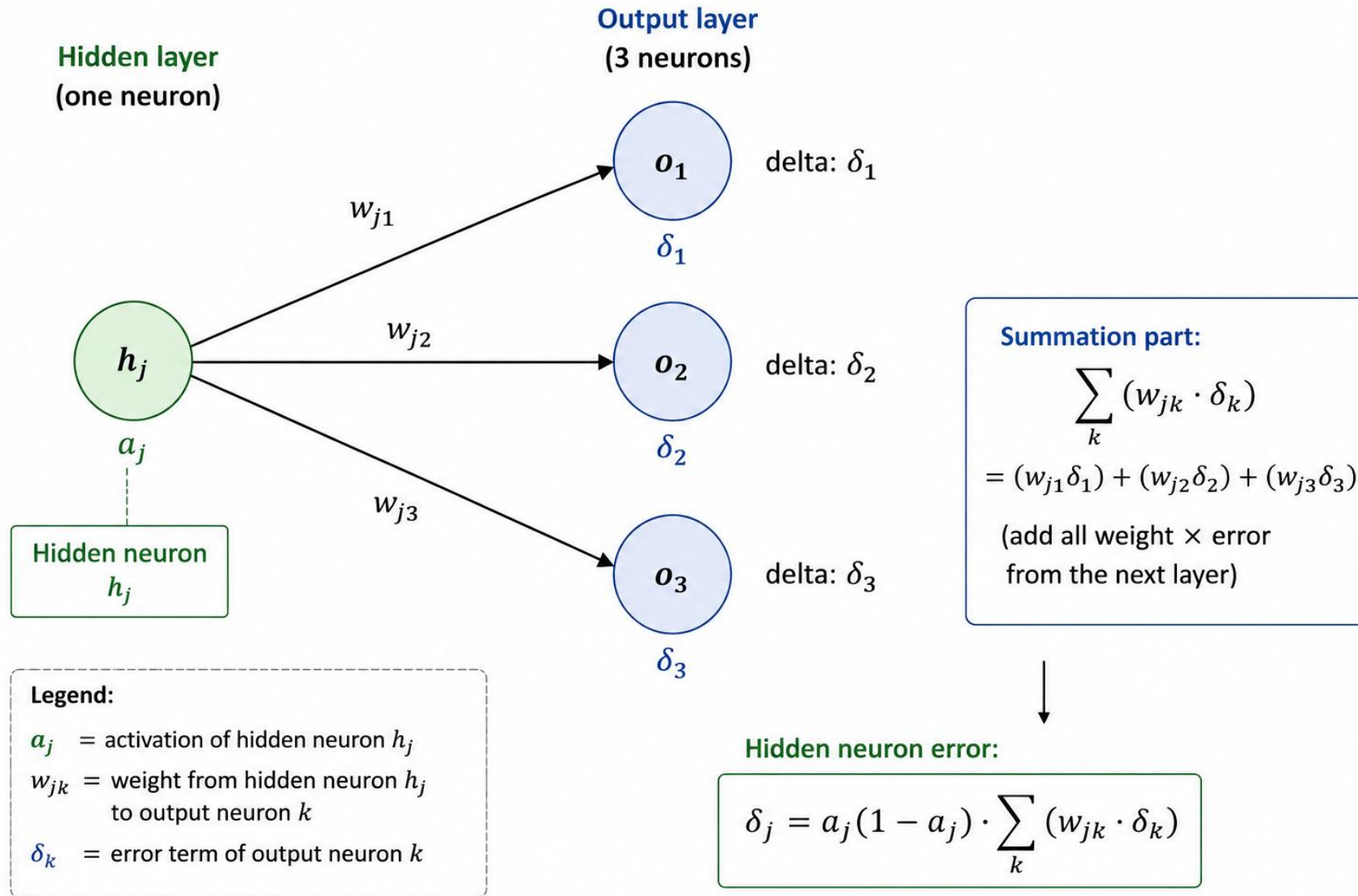
Hidden layer:

$$\delta_j = f'(z_j) \sum (w_{jk} \cdot \delta_k)$$

- If $f(z_j)$ is sigmoid:
 $\delta_j = a_j (1 - a_j) \cdot \sum (w_{jk} \cdot \delta_k)$
- If $f(z_j)$ is tanh:
 $\delta_j = (1 - a_j^2) \cdot \sum (w_{jk} \cdot \delta_k)$
- If $f(z_j)$ is ReLU:
 $\delta_j = f'(z_j) \cdot \sum (w_{jk} \cdot \delta_k)$
if $z_j > 0$: then $f'(z_j) = 1$
if $z_j \leq 0$: then $f'(z_j) = 0$

Where:

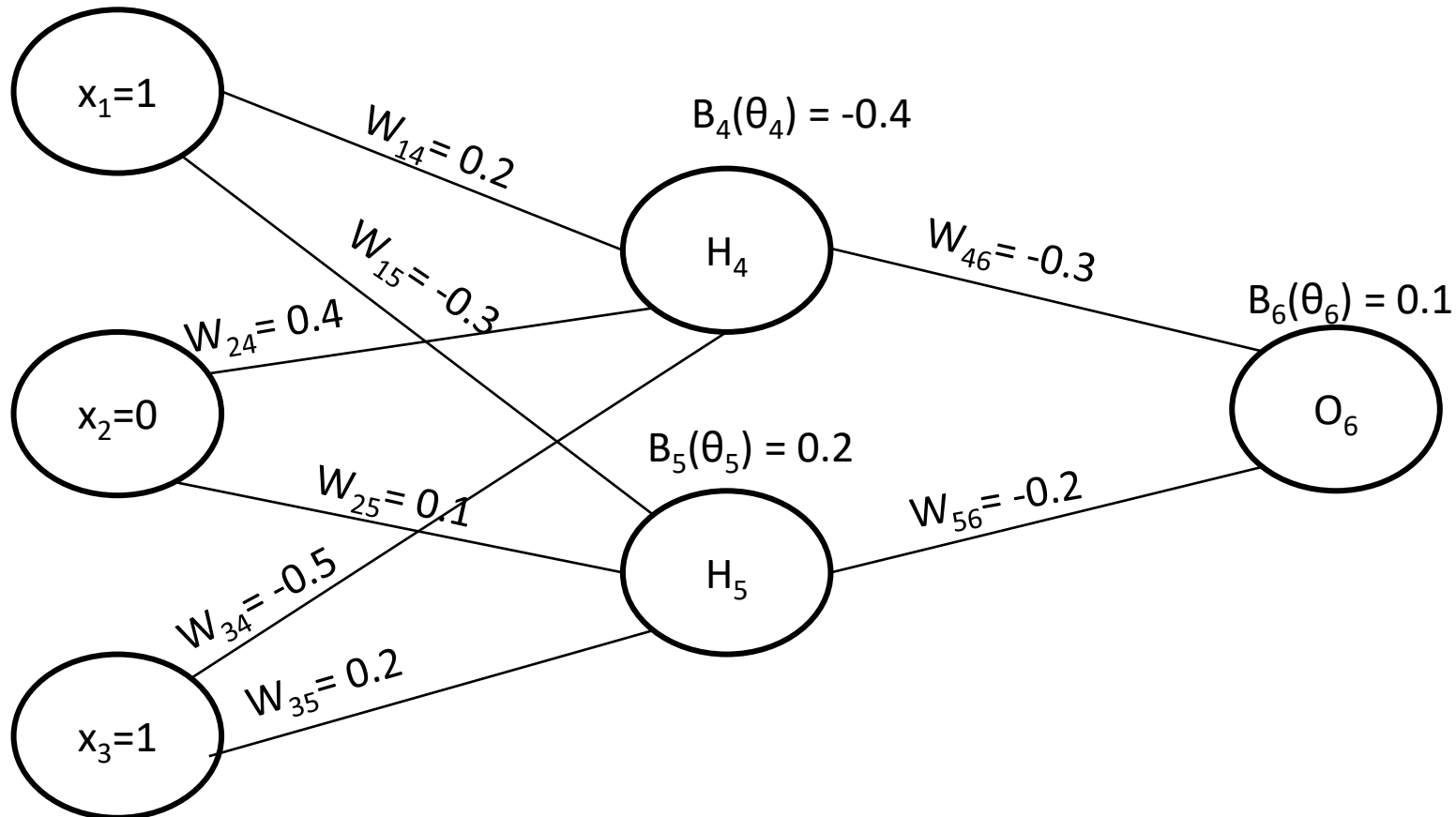
- j is the current (hidden) neuron
- k is the neuron in the next layer
- Take errors from next layer, weight them multiply by derivative of the activation function



Neral network example

Assume we have a neural network where:

- Each neuron uses a sigmoid activation function.
- We will perform a forward pass, then a backward pass, and then update the weights.
- Actual $y = 1$ and Learning rate = 0.9



Solution:

Step 1:

calculate $y_{\text{pred}}(a_j)$

$$z_j = \sum_i (w_{ij} * x_i) + b_j, \quad a_j = \frac{1}{1 + e^{a_j}}$$

$$z_4 = (w_{14} * x_1) + (w_{24} * x_2) + (w_{34} * x_3) + b_4 = ((0.2 * 1) + (0.4 * 0) + (-0.5 * 1)) + (-0.4) = -0.70$$

$$H_4 = f(z_4) = 1 / (1 + e^{-(-0.7)}) = 0.3320$$

$$z_5 = (w_{15} * x_1) + (w_{25} * x_2) + (w_{35} * x_3) + b_5 = ((-0.3 * 1) + (0.1 * 0) + (0.2 * 1)) + (0.2) = 0.1$$

$$H_5 = f(z_5) = 1 / (1 + e^{-(0.1)}) = 0.525$$

$$z_6 = (w_{46} * H_4) + (w_{56} * H_5) + b_6 = ((-0.3 * 0.332) + (-0.2 * 0.525)) + 0.1 = -0.105$$

$$O_6 = f(z_6) = 1 / (1 + e^{-(-0.105)}) = 0.4740$$

Step 2:

Calculate error (Loss)

$$L = \frac{1}{2}(y - y_{\text{pred}})^2$$

$$L = \frac{1}{2}(1 - 0.474)^2 = \frac{1}{2}(0.526)^2$$

$$L = 0.138$$

Output layer delta (δ_6):

$$\delta_6 = O_6(1 - O_6)(y - O_6)$$

$$\delta_6 = 0.474(1 - 0.474)(1 - 0.474)$$

$$\delta_6 = 0.474(1 - 0.474)(1 - 0.474)$$

$$\delta_6 = 0.474 * 0.526 * 0.526$$

$$\delta_6 = 0.131$$

Hidden layer deltas δ_4 and δ_5 :

$$\delta_4 = H_4(1 - H_4)(\delta_6 * w_{46})$$

$$\delta_4 = 0.332(1 - 0.332)(0.131 * -0.3)$$

$$\delta_4 = 0.332 * 0.668 * -0.0393$$

$$\delta_4 = -0.0087$$

$$\delta_5 = H_5(1 - H_5)(\delta_6 * w_{56})$$

$$\delta_5 = 0.525 * (1 - 0.525)(0.131 * -0.2)$$

$$\delta_5 = 0.525 * 0.475 * (-0.0262)$$

$$\delta_5 = -0.0065$$

Gradients (ΔW)

Output layer weights w_{46} and Δw_{56} :

$$\Delta w_{46} = \eta \cdot \delta_6 \cdot H_4$$

$$\Delta w_{46} = 0.9 \cdot 0.131 \cdot 0.332$$

$$\Delta w_{46} = 0.039$$

$$\Delta w_{56} = \eta \cdot \delta_6 \cdot H_5$$

$$\Delta w_{56} = 0.9 \cdot 0.131 \cdot 0.525$$

$$\Delta w_{56} = 0.062$$

Hidden layer weights w_{14} , w_{24} and w_{34} :

$$\Delta w_{14} = \eta \cdot \delta_4 \cdot X_1$$

$$\Delta w_{14} = 0.9 \cdot (-0.0087) \cdot 1$$

$$\Delta w_{14} = -0.0078$$

$$\Delta w_{24} = \eta \cdot \delta_4 \cdot X_2$$

$$\Delta w_{24} = 0.9 \cdot (-0.0087) \cdot 0$$

$$\Delta w_{24} = 0$$

$$\Delta w_{34} = \eta \cdot \delta_4 \cdot X_3$$

$$\Delta w_{34} = 0.9 \cdot (-0.0087) \cdot 1$$

$$\Delta w_{34} = -0.0078$$

Gradients (ΔW)

Hidden layer weights w_{15} , w_{25} and w_{35} :

$$\Delta w_{15} = \eta \cdot \delta_5 \cdot X_1$$

$$\Delta w_{15} = 0.9 \cdot -0.0065 \cdot 1$$

$$\Delta w_{15} = -0.0059$$

$$\Delta w_{25} = \eta \cdot \delta_5 \cdot X_2$$

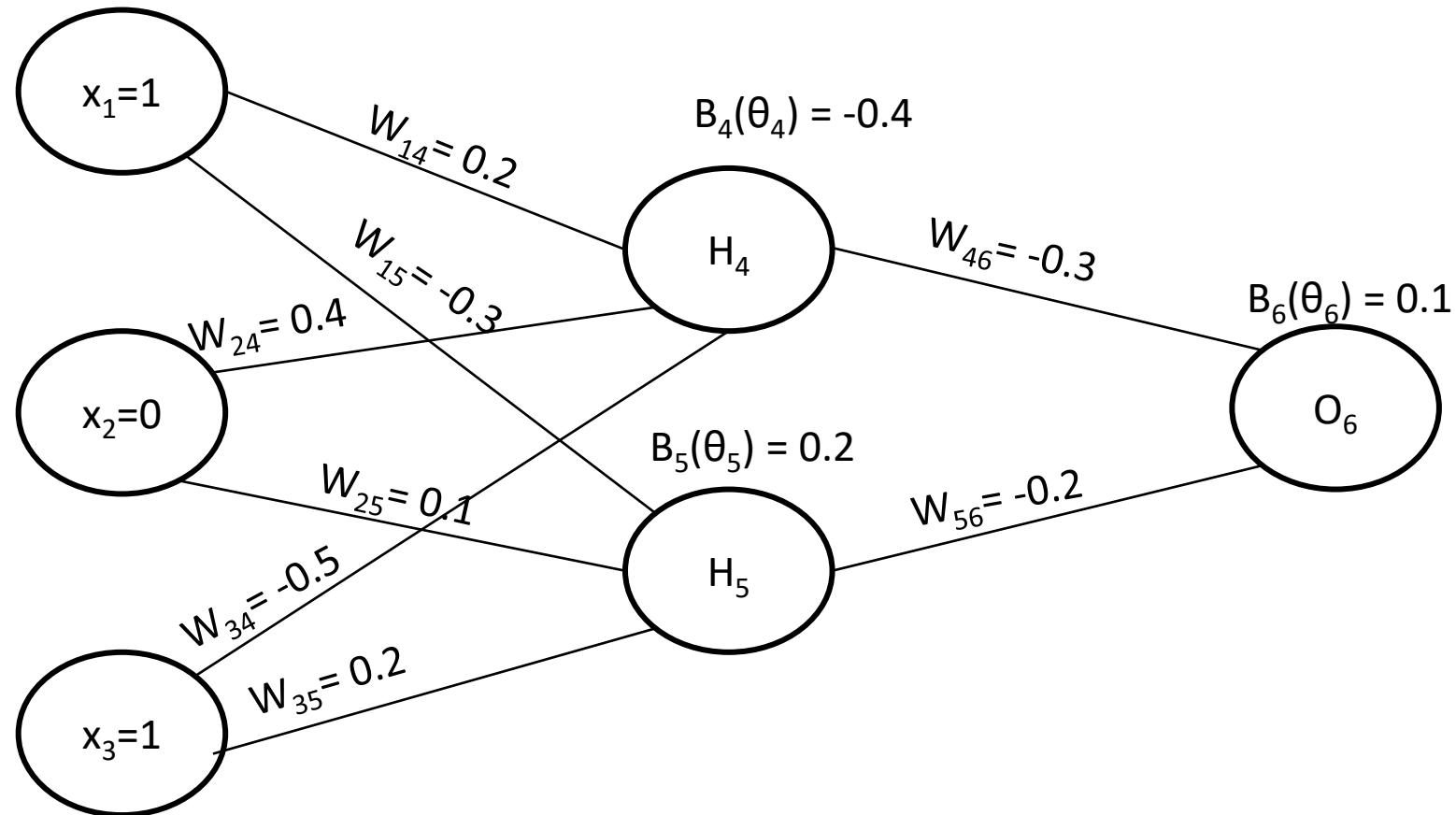
$$\Delta w_{25} = 0.9 \cdot -0.0065 \cdot 0$$

$$\Delta w_{25} = 0$$

$$\Delta w_{35} = \eta \cdot \delta_5 \cdot X_3$$

$$\Delta w_{35} = 0.9 \cdot (-0.0065) \cdot 1$$

$$\Delta w_{35} = -0.0059$$



Step 3:

Update weights and bias

Output layer w_{46_new} and w_{56_new} :

$$w_{46_new} = w_{46} + \Delta w_{46}$$

$$w_{46_new} = -0.3 + 0.039$$

$$w_{46_new} = -0.261$$

$$w_{56_new} = w_{56} + \Delta w_{56}$$

$$w_{56_new} = -0.2 + 0.062$$

$$w_{56_new} = -0.138$$

Hidden layer w_{15_new} , w_{25_new} and w_{35_new} :

$$w_{15_new} = w_{15} + \Delta w_{15}$$

$$w_{15_new} = -0.3 - 0.0059$$

$$w_{15_new} = -0.306$$

$$w_{25_new} = w_{25} + \Delta w_{25}$$

$$w_{25_new} = -0.1 + 0$$

$$w_{25_new} = 0.1$$

$$w_{35_new} = w_{35} + \Delta w_{35}$$

$$w_{35_new} = 0.2 - 0.0059$$

$$w_{35_new} = 0.194$$

Hidden layer w_{14_new} , w_{24_new} and w_{34_new} :

$$w_{14_new} = w_{14} + \Delta w_{14}$$

$$w_{14_new} = 0.2 - 0.0078$$

$$w_{14_new} = 0.192$$

$$w_{24_new} = w_{24} + \Delta w_{24}$$

$$w_{24_new} = 0.4 + 0$$

$$w_{24_new} = 0.4$$

$$w_{34_new} = w_{34} + \Delta w_{34}$$

$$w_{34_new} = -0.5 - 0.0078$$

$$w_{34_new} = -0.508$$

Output bias:

$$b_{6_new} = b_6 + \eta \cdot \delta_6$$

$$b_{6_new} = 0.1 + 0.9 \cdot 0.131$$

$$b_{6_new} = 0.218$$

Hidden bias

H5 bias:

$$b_{5_new} = b_5 + \eta \cdot \delta_5$$

$$b_{5_new} = 0.2 + 0.9 \cdot -0.0065$$

$$b_{5_new} = 0.194$$

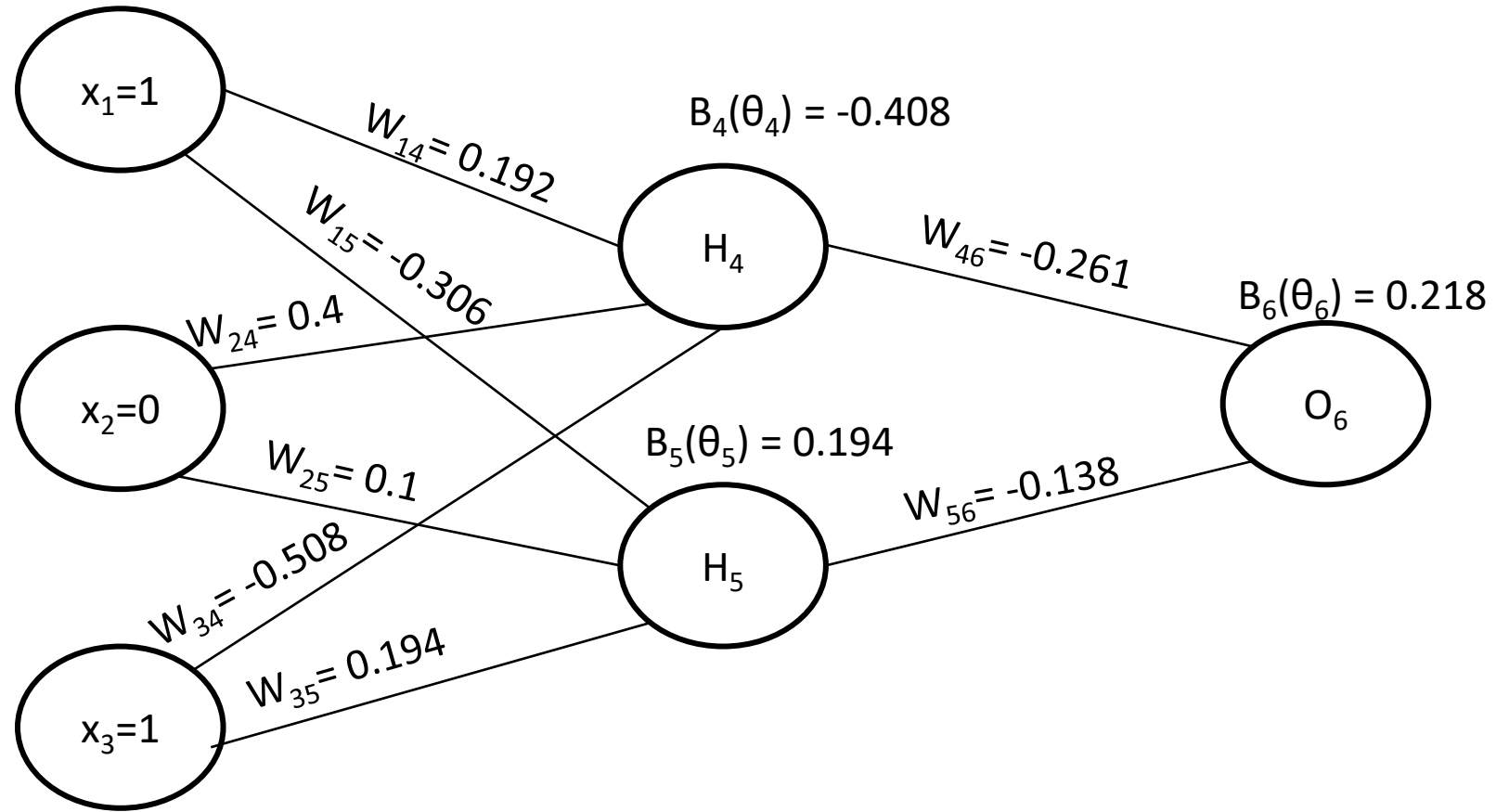
H4 bias:

$$b_{4_new} = b_4 + \eta \cdot \delta_4$$

$$b_{4_new} = -0.4 + 0.9 \cdot -0.0087$$

$$b_{4_new} = -0.408$$

Updated neural network



Neural Network Classification

Example 1:

- Using `lode_digits` dataset that, consists of 1,797 samples, where each sample is represented as a fixed-size 8×8 matrix.
- There are 10 possible classes or labels (0, 1, 2, 3, ..., 9).
- Each class contains approximately 180 samples, and the features are integer numbers between 0 and 16.
- This dataset is available in Scikit-learn.
- In this exercise, you will implement a neural network using the Python library Keras to train and test a model for the automatic recognition of handwritten.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

images=load_digits()
images.data.shape
images.target.shape

plt.figure(figsize=(20,8))
for index,(image,label) in enumerate(zip(images.data[0:5],images.target[0:5])):
plt.subplot(1,5,index+1)
plt.imshow(np.reshape(image,(8,8)),cmap=plt.cm.gray_r)
plt.title(f"Training {label}")
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test =
train_test_split(samples,labels,test_size=0.2,random_state=100)
```

```
from keras.models import Sequential
from keras.layers import Dense
# Define the neural network model
model = Sequential()
model.add(Dense(512, activation='relu', input_shape=(x_train.shape[1],)))
model.add(Dense(256, activation='relu'))
model.add(Dense(128, activation='relu'))

model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(16, activation='relu'))
model.add(Dense(10, activation='softmax'))
# Compile the model
model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
# Train the model
model.fit(x_train, y_train, epochs=10, batch_size=200,validation_split=0.2)
# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test)
print('Test accuracy:', test_acc)
```

Neural Network Classification

Example 2:

Implement a Neural Network Regression model using tensorflow library in python and the following dataset (insurance.csv)

age	sex	bmi	children	smoker	region	charges
19	female	27.900	0	yes	southwest	16884.92400
18	male	33.770	1	no	southeast	1725.55230
28	male	33.000	3	no	southeast	4449.46200
33	male	22.705	0	no	northwest	21984.47061
32	male	28.880	0	no	northwest	3866.85520

Code

Importing the libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Importing the dataset

```
df = pd.read_csv("/content/drive/MyDrive/AI course/insurance.csv")
df.head()
```

Analysis the dataset

```
df.dtypes
df.nunique()
df.isna().sum()
```

Encoding categorical data

```
df = pd.get_dummies(df, drop_first=True)
df.shape
```

Data Visualization Building the Correlation matrix

```
import seaborn as sns
plt.figure(figsize=(12,10))
sns.heatmap(df.corr(), annot = True, cmap="Blues")
```

#Extracting the Independent and Dependent variables

```
x = df.drop("charges",axis=1)
y = df["charges"]
x.head()
y.head()
```

Splitting the dataset into the Training set and Test set

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,
random_state=2)
```

Built the model and Fitting the Training set

```
import tensorflow as tf
model = tf.keras.Sequential([
    tf.keras.layers.Dense(256, activation='relu',
        input_shape=(x_train.shape[1],)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(1)])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.fit(x_train, y_train, epochs=100, validation_split=0.2)
```

Model evaluation

```
loss, mae = model.evaluate(x_test, y_test)
print("Loss (MSE):", loss)
print("MAE:", mae)

from sklearn.metrics import r2_score
y_pred = model.predict(x_test)
print("R2 Score:", r2_score(y_test, y_pred))
```